

PROGRAMMATION EN LANGAGE C

Licence 1 - Mathématiques, Physique et Chimie

Résumé du cours

1. Structure d'un programme C.

```

1 #include <stdio.h>
2 int main() {
3     /* instructions */
4     return 0;
5 }
```

2. Types de base. `int` (entier), `float` (flottant simple précision), `double` (double précision), `char` (caractère). Déclaration : `type nom = valeur;`.

3. Entrées/Sorties. `printf("%d %f %c %s", i, x, c, s);` affiche entier, flottant, caractère, chaîne. `scanf("%d", &i);` lit un entier.

4. Structures de contrôle.

- . Conditionnelle : `if (cond) {...} else {...}`
- . Boucle `for` : `for (init; cond; increment) {...}`
- . Boucle `while` : `while (cond) {...}`
- . Boucle `do...while` : `do {...} while (cond);`

5. Tableaux. `int tab[10];` déclare un tableau de 10 entiers. Accès : `tab[i]` pour l'indice i (de 0 à $n - 1$).

6. Fonctions. `type_retour nom(params) {...; return val;}`. Passage par valeur (copie) ou par pointeur (modification de l'original).

7. Pointeurs. `int *p = &a;` ; p contient l'adresse de a . `*p` accède à la valeur pointée. `p++` passe à l'élément suivant.

1. Bases du langage C

Exercice 1 - Variables et opérations

★

Exercice

Écrire un programme C qui :

1. Demande à l'utilisateur de saisir deux entiers a et b .
2. Calcule et affiche leur somme, différence, produit et quotient entier.
3. Affiche également le reste de la division entière de a par b .

```

1 #include <stdio.h>
2
3 int main() {
4     int a, b;
5
6     printf("Entrez deux entiers a et b : ");
7     scanf("%d %d", &a, &b);
8
9     printf("Somme      : %d + %d = %d\n", a, b, a + b);
10    printf("Difference : %d - %d = %d\n", a, b, a - b);
11    printf("Produit    : %d * %d = %d\n", a, b, a * b);
12
13    if (b != 0) {
14        printf("Quotient   : %d / %d = %d\n", a, b, a / b);
15        printf("Reste      : %d %% %d = %d\n", a, b, a % b);
16    } else {
17        printf("Erreur : division par zero impossible.\n");
18    }
19
20    return 0;
21 }
```

Points importants :

- . En C, la division entière a/b tronque vers zéro.
- . L'opérateur `%` donne le reste de la division euclidienne.
- . Il faut vérifier que $b \neq 0$ avant de diviser.
- . `scanf` prend l'adresse des variables avec `&`.
- . Pour afficher le caractère `%` avec `printf`, on écrit `%%`.

Exercice 2 - Structures conditionnelles

★

Exercice

Écrire un programme C qui :

1. Lit un entier n .
2. Affiche si n est positif, négatif ou nul.
3. Détermine si n est pair ou impair.
4. Affiche la valeur absolue de n .

```

1 #include <stdio.h>
2
3 int main() {
4     int n;
5
6     printf("Entrez un entier : ");
7     scanf("%d", &n);
8
9     /* Signe */
10    if (n > 0) {
11        printf("%d est positif.\n", n);
12    } else if (n < 0) {
13        printf("%d est negatif.\n", n);
14    } else {
15        printf("Le nombre est nul.\n");
16    }
17
18    /* Parite */
19    if (n % 2 == 0) {
20        printf("%d est pair.\n", n);
21    } else {
22        printf("%d est impair.\n", n);
23    }
24
25    /* Valeur absolue */
26    int abs_n = (n >= 0) ? n : -n;
27    printf("Valeur absolue de %d : %d\n", n, abs_n);
28
29    return 0;
30 }
```

L'opérateur ternaire (cond) ? val_vrai : val_faux est une façon concise d'écrire

un if-else sur une expression.

2. Boucles et tableaux

Exercice 3 - Boucles et calculs

★★

Exercice

Écrire des programmes C pour :

- (a) Calculer la factorielle $n!$ pour un n saisi par l'utilisateur.
- (b) Afficher la table de multiplication de 1 à 10 pour un entier k donné.
- (c) Calculer la somme $S = 1 + 2 + \dots + n$ et vérifier avec la formule $n(n+1)/2$.
- (d) Déterminer si un entier n est premier.

(a) Factorielle :

```

1 #include <stdio.h>
2 int main() {
3     int n;
4     long long fact = 1;
5     printf("n = "); scanf("%d", &n);
6     for (int i = 1; i <= n; i++) {
7         fact *= i;
8     }
9     printf("%d! = %lld\n", n, fact);
10    return 0;
11 }
```

(b) Table de multiplication :

```

1 int k;
2 printf("k = "); scanf("%d", &k);
3 for (int i = 1; i <= 10; i++) {
4     printf("%d x %d = %d\n", k, i, k * i);
5 }
```

(c) Somme avec vérification :

```

1 int n, S = 0;
2 printf("n = "); scanf("%d", &n);
3 for (int i = 1; i <= n; i++) S += i;
4 printf("Somme boucle : %d\n", S);
```

```
5 printf("Formule n(n+1)/2 : %d\n", n * (n + 1) / 2);
```

(d) Test de primalité :

```
1 #include <stdio.h>
2 int est_premier(int n) {
3     if (n < 2) return 0;
4     for (int i = 2; i * i <= n; i++) {
5         if (n % i == 0) return 0; /* divisible */
6     }
7     return 1;
8 }
9 int main() {
10     int n;
11     printf("n = "); scanf("%d", &n);
12     if (est_premier(n))
13         printf("%d est premier.\n", n);
14     else
15         printf("%d n'est pas premier.\n", n);
16     return 0;
17 }
```

L'optimisation clé : on teste les diviseurs jusqu'à $\sqrt{n}$ seulement.

Exercice 4 - Tableaux et tri

★★

Exercice

1. Écrire un programme qui remplit un tableau de n entiers, puis calcule et affiche la somme, la moyenne, le maximum et le minimum.
2. Implémenter le tri à bulles (*bubble sort*) pour trier un tableau dans l'ordre croissant.

1.

```
1 #include <stdio.h>
2 #define N 10
3 int main() {
4     int tab[N], somme = 0, max, min;
5     printf("Entrez %d entiers :\n", N);
6     for (int i = 0; i < N; i++) scanf("%d", &tab[i]);
7
8     max = min = tab[0];
```

```

9     for (int i = 0; i < N; i++) {
10         somme += tab[i];
11         if (tab[i] > max) max = tab[i];
12         if (tab[i] < min) min = tab[i];
13     }
14     printf("Somme    = %d\n", somme);
15     printf("Moyenne = %.2f\n", (double)somme / N);
16     printf("Max      = %d\n", max);
17     printf("Min      = %d\n", min);
18     return 0;
19 }
```

2. Tri à bulles :

```

1 void tri_bulles(int tab[], int n) {
2     int tmp;
3     for (int i = 0; i < n - 1; i++) {
4         for (int j = 0; j < n - 1 - i; j++) {
5             if (tab[j] > tab[j + 1]) {
6                 /* echange */
7                 tmp = tab[j];
8                 tab[j] = tab[j + 1];
9                 tab[j + 1] = tmp;
10            }
11        }
12    }
13 }
```

Le tri à bulles a une complexité $O(n^2)$: à chaque passage, le plus grand élément non trié remonte à sa place (comme une bulle).

3. Fonctions et pointeurs

Exercice 5 - Fonctions

★★

Exercice

1. Écrire une fonction `puissance(double base, int n)` qui calcule $base^n$ par multiplications successives.
2. Écrire une fonction récursive `fibonacci(int n)` qui calcule le n -ième terme de la suite de Fibonacci définie par $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$.
3. Écrire une fonction `pgcd(int a, int b)` qui calcule le PGCD par l'algorithme

d'Euclide.

```

1 #include <stdio.h>
2
3 /* (1) Puissance iterative */
4 double puissance(double base, int n) {
5     double resultat = 1.0;
6     int exp = (n >= 0) ? n : -n;
7     for (int i = 0; i < exp; i++) resultat *= base;
8     return (n >= 0) ? resultat : 1.0 / resultat;
9 }
10
11 /* (2) Fibonacci recursif */
12 long long fibonacci(int n) {
13     if (n == 0) return 0;
14     if (n == 1) return 1;
15     return fibonacci(n - 1) + fibonacci(n - 2);
16 }
17
18 /* (3) PGCD par algorithme d'Euclide */
19 int pgcd(int a, int b) {
20     while (b != 0) {
21         int r = a % b;
22         a = b;
23         b = r;
24     }
25     return a;
26 }
27
28 int main() {
29     printf("2^10 = %.0f\n", puissance(2, 10));
30     printf("F(10) = %lld\n", fibonacci(10));
31     printf("pgcd(48, 18) = %d\n", pgcd(48, 18));
32     return 0;
33 }

```

Attention : la version récursive de Fibonacci est très inefficace pour les grands n (complexité exponentielle). En pratique, on utilise une version itérative ou la mémoïsation.

Exercice 6 - Pointeurs et passage par adresse

★★★

Exercice

1. Écrire une fonction `echange(int *a, int *b)` qui échange les valeurs de deux variables. Expliquer pourquoi le passage par valeur ne fonctionnerait pas.
2. Écrire une fonction qui calcule simultanément le minimum et le maximum d'un tableau et les retourne via des pointeurs.
3. Qu'affiche le programme suivant ?

```

1  int main() {
2      int t[] = {10, 20, 30, 40, 50};
3      int *p = t;
4      printf("%d\n", *p);
5      printf("%d\n", *(p + 2));
6      p++;
7      printf("%d\n", *p);
8      printf("%d\n", p[2]);
9      return 0;
10 }
```

1.

```

1  void echange(int *a, int *b) {
2      int tmp = *a;
3      *a = *b;
4      *b = tmp;
5  }
```

Si on passait par valeur (`void echange(int a, int b)`), la fonction travaillerait sur des copies locales ; les variables originales dans `main` resteraient inchangées. Avec les pointeurs, on accède directement aux emplacements mémoire d'origine.

2.

```

1  void min_max(int tab[], int n, int *min, int *max) {
2      *min = *max = tab[0];
3      for (int i = 1; i < n; i++) {
4          if (tab[i] < *min) *min = tab[i];
5          if (tab[i] > *max) *max = tab[i];
6      }
7  }
8  /* Appel : int mn, mx; min_max(t, 5, &mn, &mx); */
```

3. Analyse pas à pas :


```

. p = t : p pointe sur t[0] (valeur 10).
. *p : valeur à l'adresse p = 10.
. *(p+2) : adresse de t[2], valeur = 30.
. p++ : p avance d'un entier, pointe maintenant sur t[1] (valeur 20).
. *p = 20.
. p[2] ≡ *(p+2) : deux cases après t[1], soit t[3] = 40.
    
```

Le programme affiche : 10, 30, 20, 40.

Exercice 7 - Récursivité : suite de Fibonacci

**

Exercice

La suite de Fibonacci est définie par $F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$ pour $n \geq 2$.

1. Écrire une fonction C récursive `fib_rec(int n)` qui calcule F_n .
2. Écrire une fonction C itérative `fib_iter(int n)` qui calcule F_n .
3. Comparer les complexités temporelles des deux approches. Pourquoi la version récursive est-elle très lente pour n grand ?
4. Écrire un programme `main` qui affiche les 15 premiers termes avec les deux méthodes.
5. Modifier `fib_iter` pour qu'elle calcule également la somme $S_n = \sum_{k=0}^n F_k$.

1. Version récursive :

```

1 int fib_rec(int n) {
2     if (n <= 0) return 0;
3     if (n == 1) return 1;
4     return fib_rec(n-1) + fib_rec(n-2);
5 }
    
```

2. Version itérative :

```

1 int fib_iter(int n) {
2     if (n <= 0) return 0;
3     if (n == 1) return 1;
4     int a = 0, b = 1, c;
5     for (int i = 2; i <= n; i++) {
6         c = a + b;
7         a = b;
8         b = c;
9     }
10    return b;
11 }
    
```

3. Complexité :

- . **fib_rec** : complexité $O(\varphi^n)$ où $\varphi = (1 + \sqrt{5})/2 \approx 1,618$ (nombre d'or). Chaque appel engendre 2 sous-appels; de nombreux sous-problèmes sont recalculés plusieurs fois (arbre d'appels exponentiel).
- . **fib_iter** : complexité $O(n)$ en temps, $O(1)$ en espace. Elle est donc exponentiellement plus rapide.

Pour $n = 40$, **fib_rec** effectue $\approx 10^8$ appels tandis que **fib_iter** effectue 40 itérations.

4. Programme principal :

```

1 #include <stdio.h>
2 int main(void) {
3     printf("n    fib_rec    fib_iter\n");
4     for (int i = 0; i < 15; i++)
5         printf("%2d    %6d    %6d\n", i, fib_rec(i), fib_iter(i));
6     return 0;
7 }
```

5. Calcul simultané de la somme ($S_n = F_{n+2} - 1$) :

```

1 int fib_iter_sum(int n, int *somme) {
2     if (n <= 0) { *somme = 0; return 0; }
3     int a = 0, b = 1, c, s = 1;
4     for (int i = 2; i <= n; i++) {
5         c = a + b; a = b; b = c; s += c;
6     }
7     *somme = s;
8     return b;
9 }
```

Exercice 8 - Tableaux 2D et algèbre matricielle

★★

Exercice

On travaille avec des matrices 4×4 stockées comme tableaux 2D en C.

1. Écrire une fonction `init_identite(int M[4][4])` qui initialise M à la matrice identité.
2. Écrire une fonction `trace(int M[4][4])` qui retourne la trace de M .
3. Écrire une fonction `transposee(int A[4][4], int B[4][4])` qui stocke dans B la transposée de A .

4. Écrire une fonction `produit_mat_vec(int A[4][4], int v[4], int res[4])` qui calcule $res = A\vec{v}$.
5. Tester avec la matrice $A_{ij} = i + j$ (i, j de 0 à 3) et le vecteur $\vec{v} = (1, 0, 0, 0)^T$.

1.

```
1 void init_identite(int M[4][4]) {
2     for (int i = 0; i < 4; i++)
3         for (int j = 0; j < 4; j++)
4             M[i][j] = (i == j) ? 1 : 0;
5 }
```

2.

```
1 int trace(int M[4][4]) {
2     int t = 0;
3     for (int i = 0; i < 4; i++) t += M[i][i];
4     return t;
5 }
```

3.

```
1 void transposee(int A[4][4], int B[4][4]) {
2     for (int i = 0; i < 4; i++)
3         for (int j = 0; j < 4; j++)
4             B[j][i] = A[i][j];
5 }
```

4.

```
1 void produit_mat_vec(int A[4][4], int v[4], int res[4]) {
2     for (int i = 0; i < 4; i++) {
3         res[i] = 0;
4         for (int j = 0; j < 4; j++)
5             res[i] += A[i][j] * v[j];
6     }
7 }
```

5. Pour $A_{ij} = i + j$: $A = \begin{pmatrix} 0 & 1 & 2 & 3 \\ 1 & 2 & 3 & 4 \\ 2 & 3 & 4 & 5 \\ 3 & 4 & 5 & 6 \end{pmatrix}$. Trace : $0 + 2 + 4 + 6 = 12$. Produit $A\vec{v}$

avec $\vec{v} = (1, 0, 0, 0)^T$ donne la première colonne de A : $(0, 1, 2, 3)^T$.

```

1 int main(void) {
2     int A[4][4], v[4]={1,0,0,0}, res[4], B[4][4];
3     for (int i=0;i<4;i++) for(int j=0;j<4;j++) A[i][j]=i+j;
4     printf("Trace = %d\n", trace(A));          /* 12 */
5     produit_mat_vec(A, v, res);
6     printf("Av = (%d,%d,%d,%d)\n", res[0],res[1],res[2],res[3])
7     ;
8     return 0;
9 }

```

Exercice 9 - Allocation dynamique et pointeurs sur fonctions ***

Exercice

1. Écrire une fonction `swap(int *a, int *b)` qui échange les valeurs de deux variables entières. Expliquer pourquoi le passage par pointeur est nécessaire.
2. Écrire une fonction `creer_tableau(int n)` qui alloue dynamiquement un tableau de n entiers, les initialise à $0, 1, 2, \dots, n-1$ et retourne le pointeur.
3. Écrire une fonction `appliquer(int *t, int n, int (*f)(int))` qui applique la fonction f à chaque élément du tableau.
4. Tester avec les fonctions $f(x) = x^2$ et $g(x) = 2x+1$ sur un tableau de 6 éléments.
5. Expliquer ce que fait le code suivant et identifier l'erreur :

```
int *p = malloc(10*sizeof(int)); p[10] = 5;
```

1.

```

1 void swap(int *a, int *b) {
2     int tmp = *a;
3     *a = *b;
4     *b = tmp;
5 }

```

Sans pointeurs, C passe les arguments par valeur : la fonction travaillerait sur des copies et les variables originales resteraient inchangées.

2.

```

1 int *creer_tableau(int n) {
2     int *t = malloc(n * sizeof(int));
3     if (t == NULL) { fprintf(stderr, "Erreur malloc\n"); exit
4         (1); }
5     for (int i = 0; i < n; i++) t[i] = i;

```

```

5     return t;
6 }
    
```

Remarque : le tableau alloué doit être libéré par l'appelant avec `free(t)`.

3.

```

1 void appliquer(int *t, int n, int (*f)(int)) {
2     for (int i = 0; i < n; i++) t[i] = f(t[i]);
3 }
    
```

4.

```

1 int carre(int x) { return x * x; }
2 int double_plus_un(int x) { return 2*x + 1; }
3
4 int main(void) {
5     int *t = creer_tableau(6); /* {0,1,2,3,4,5} */
6     appliquer(t, 6, carre);    /* {0,1,4,9,16,25} */
7     for (int i=0;i<6;i++) printf("%d ", t[i]);
8     free(t);
9     t = creer_tableau(6);
10    appliquer(t, 6, double_plus_un); /* {1,3,5,7,9,11} */
11    for (int i=0;i<6;i++) printf("%d ", t[i]);
12    free(t);
13    return 0;
14 }
    
```

5. `malloc(10*sizeof(int))` alloue 10 entiers aux indices 0 à 9. `p[10]=5` écrit en dehors de la zone allouée (**dépassement de tampon** – *buffer overflow*). C'est un comportement indéfini qui peut provoquer une corruption mémoire, un segfault ou des failles de sécurité.

Exercice 10 - Lecture de fichier et statistiques

★★

Exercice

Un fichier texte `donnees.txt` contient des entiers séparés par des espaces ou sauts de ligne.

1. Écrire une fonction `lire_fichier(const char *nom, int *t, int max)` qui lit les entiers du fichier dans le tableau `t` (de taille maximale `max`) et retourne le nombre de valeurs lues.
2. Écrire une fonction `moyenne(int *t, int n)` qui retourne la moyenne sous forme de `double`.

3. Écrire une fonction `ecart_type(int *t, int n, double moy)` qui retourne l'écart-type $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2}$.
4. Écrire le programme principal qui affiche n , $\bar{x}$, σ_x , le minimum et le maximum.

1.

```
1 #include <stdio.h>
2 #include <math.h>
3
4 int lire_fichier(const char *nom, int *t, int max) {
5     FILE *f = fopen(nom, "r");
6     if (f == NULL) { perror("fopen"); return -1; }
7     int n = 0;
8     while (n < max && fscanf(f, "%d", &t[n]) == 1) n++;
9     fclose(f);
10    return n;
11 }
```

2.

```
1 double moyenne(int *t, int n) {
2     double s = 0.0;
3     for (int i = 0; i < n; i++) s += t[i];
4     return s / n;
5 }
```

3.

```
1 double ecart_type(int *t, int n, double moy) {
2     double s = 0.0;
3     for (int i = 0; i < n; i++) {
4         double d = t[i] - moy;
5         s += d * d;
6     }
7     return sqrt(s / n);
8 }
```

4.

```
1 int main(void) {
2     int t[1000];
3     int n = lire_fichier("donnees.txt", t, 1000);
4     if (n <= 0) return 1;
5     double moy = moyenne(t, n);
```

```

6     double sigma = ecart_type(t, n, moy);
7     int mn = t[0], mx = t[0];
8     for (int i = 1; i < n; i++) {
9         if (t[i] < mn) mn = t[i];
10        if (t[i] > mx) mx = t[i];
11    }
12    printf("n      = %d\n", n);
13    printf("moy    = %.4f\n", moy);
14    printf("sigma  = %.4f\n", sigma);
15    printf("min    = %d, max = %d\n", mn, mx);
16    return 0;
17 }

```

Exercice 11 - Structures et tri de points

★★

Exercice

On souhaite manipuler des points du plan.

1. Définir une structure `Point` avec deux champs `double x` et `double y`.
2. Écrire une fonction `distance(Point a, Point b)` qui retourne la distance euclidienne entre deux points.
3. Écrire une fonction de comparaison `cmp_x` utilisable avec `qsort` pour trier un tableau de points par abscisse croissante.
4. Dans `main`, créer un tableau de 5 points, le trier par abscisse avec `qsort`, puis afficher les points triés.
5. Écrire une fonction `barycentre(Point *t, int n)` qui retourne le barycentre de n points.

1.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <math.h>
4
5 typedef struct {
6     double x, y;
7 } Point;

```

2.

```

1 double distance(Point a, Point b) {

```

```

2     double dx = a.x - b.x;
3     double dy = a.y - b.y;
4     return sqrt(dx*dx + dy*dy);
5 }
    
```

3. La fonction de comparaison pour qsort reçoit des pointeurs génériques void* :

```

1 int cmp_x(const void *p1, const void *p2) {
2     const Point *a = (const Point *)p1;
3     const Point *b = (const Point *)p2;
4     if (a->x < b->x) return -1;
5     if (a->x > b->x) return 1;
6     return 0;
7 }
    
```

4.

```

1 int main(void) {
2     Point pts[5] = {{3,1},{1,4},{4,1},{1,5},{9,2}};
3     qsort(pts, 5, sizeof(Point), cmp_x);
4     for (int i = 0; i < 5; i++)
5         printf("(%.1f, %.1f)\n", pts[i].x, pts[i].y);
6     /* Affiche dans l'ordre des x : 1,1,3,4,9 */
7     return 0;
8 }
    
```

5.

```

1 Point barycentre(Point *t, int n) {
2     Point g = {0.0, 0.0};
3     for (int i = 0; i < n; i++) {
4         g.x += t[i].x;
5         g.y += t[i].y;
6     }
7     g.x /= n;
8     g.y /= n;
9     return g;
10 }
    
```

Pour les 5 points ci-dessus, le barycentre est $G = (18/5, 13/5) = (3,6; 2,6)$.